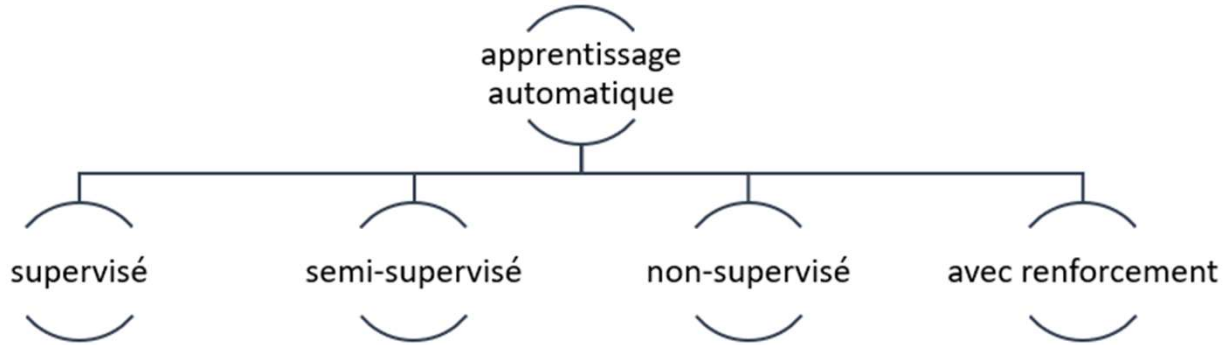
Réseaux de neurones artificiels et optimisation de l'apprentissage

COMMENT FONCTIONNE L'ALGORITHME DE DESCENTE DE GRADIENT ET EN QUOI PERMET-IL L'APPRENTISSAGE SUPERVISÉ ?

Intelligence artificielle et apprentissage automatique



Type d'apprentissage	Tâches principales	Exemples d'utilisation
Apprentissage supervisé	– La classification : prédire une catégorie (ex : spam ou non spam) – La régression : prédire une valeur numérique (ex : le prix d'une maison)	– Les arbres de décision – Les machines à vecteurs de support (SVM) – La régression logistique – Les réseaux de neurones artificiels

→ Modèle du perceptron

https://www.learnthings.fr/quelles-sont-les-branches-de-lintelligence-artificielle/#:~:text=Nous%20explorerons%20en%20d%C3%A9tail%20les,et%20leurs%20sous%2Ddomaines%20cl%C3%A9s.

SOMMAIRE

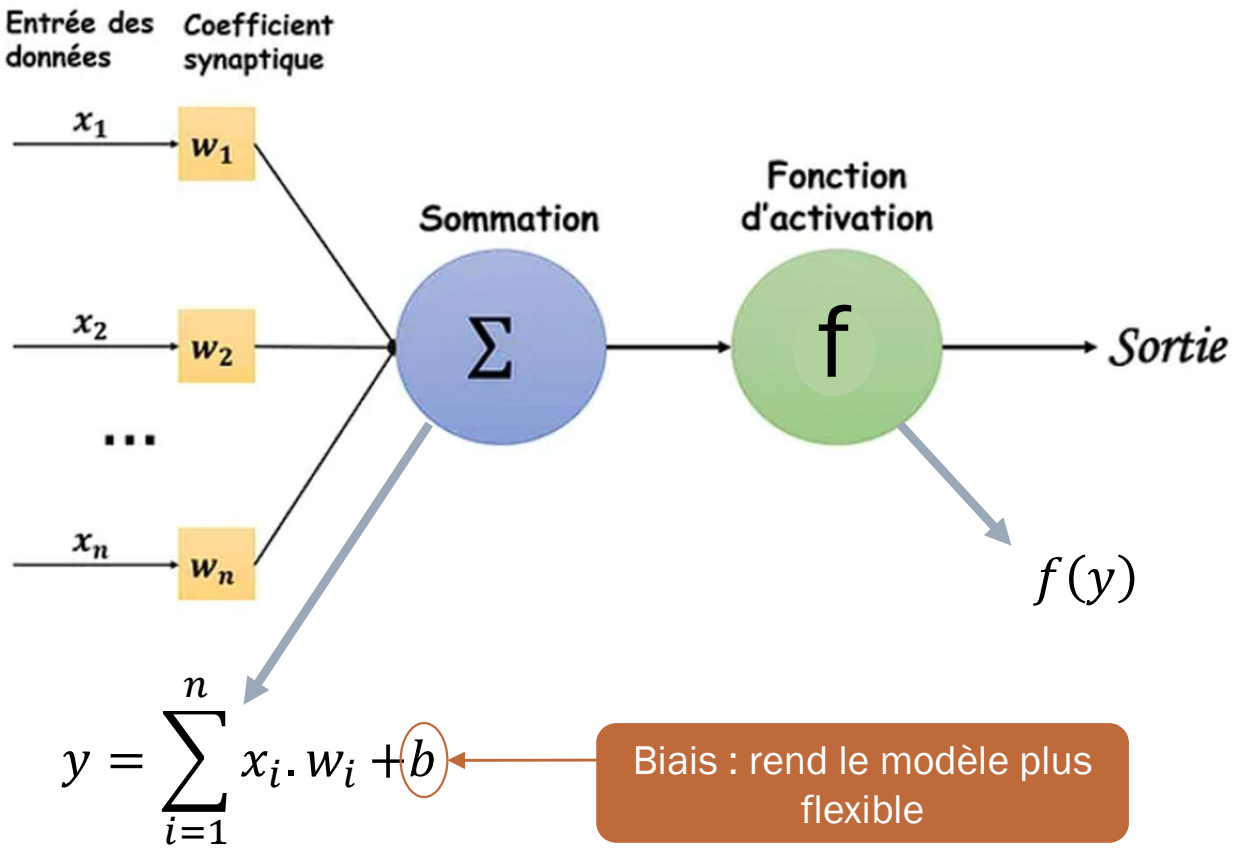
I. Introduction au modèle
   i. Le perceptron
   ii. Son apprentissage

II. Régression linéaire
   i. Algorithme du gradient
   ii. Générer un nuage de points
   iii. Résultats de la programmation
   iv. Améliorations et limites

III. Apprentissage en dimension 2
   i. Algorithme du gradient (adapté)
   ii. Générer un nuage de points (3D)
   iii. Résultats de la programmation

ANNEXES

Qu'est-ce qu'un perceptron ?



https://medium.com/@VeritasLuminis/tout-ce-dont-vous-devez-savoir-sur-le-mod%C3%A8le-du-perceptron-74da4df52712

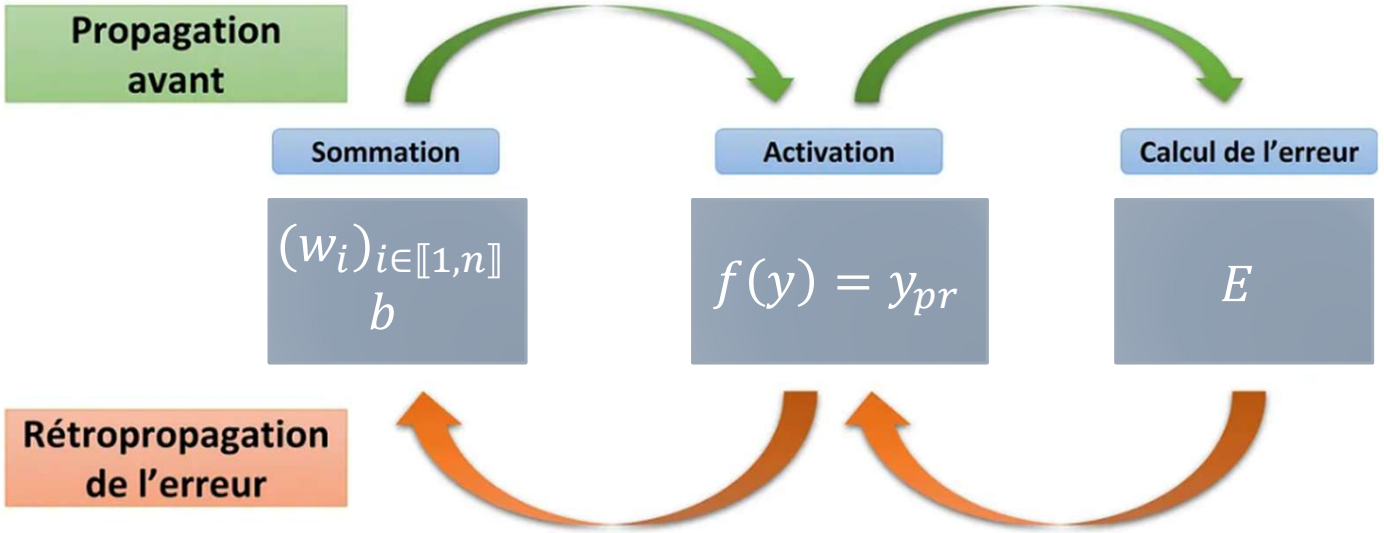
Apprentissage du perceptron

Fonction Erreur :

$$E = \frac{1}{n}\sum_{i=1}^{n}\left(y_{pr(i)} - y_{th(i)}\right)^2$$

OBJECTIF: minimiser l'erreur

$$y_{pr(i)} = f(x_i . w_i + b)$$



Régression linéaire
$ax + b$

Données d'apprentissage :

$$(x_i, y_i)_{i \in [\![1,n]\!]}$$

Coefficient synaptique ou poids :

W

Fonction d'activation :

f = id

Modèle prédit :

$$y_{pr} = (w.\, x_1 + b, \ldots,\ \mathrm{w}.\, x_n + b)$$

Modèle théorique :

$$y_{th} = (y_1, \ldots, y_n)$$

a : coefficient directeur
b: ordonnée à l'origine

Algorithme de descente du gradient

Dérivées partielles de l'Erreur :

$$\frac{\partial E}{\partial w} = \frac{2}{n}\sum_{i=1}^{n}\left(y_{pr(i)} - y_{th(i)}\right).x_i$$

$$\frac{\partial E}{\partial b} = \frac{2}{n}\sum_{i=1}^{n}\left(y_{pr(i)} - y_{th(i)}\right)$$

$$y_{pr} = (w.x_1 + b,\ \text{w}.x_2 + b\ , \dots\ ,\ \text{w}.x_n + b)$$

$$y_{th} = (y_1\ , \dots , y_n)$$

Modification des paramètres :

À l'itération N+1 :

- $w_{N+1} = w_N - \mu\frac{\partial E}{\partial w}$
- $b_{N+1} = b_N - \mu\frac{\partial E}{\partial b}$

Taux apprentissage

Paramètres générés aléatoirement

```
#modèle non entrainé
np.random.seed(0)
w=np.random.random()
b=np.random.random()
print('paramètres générés (w,b)=(',w,',',b,')')
```

$x = [liste\ d'abscisses]$ $y = [liste\ de\ valeurs\ théoriques]$

```
N=1000 #nbr de fois que le programme est entrainé
μ=0.02  #taux d'apprentissage
```

$$\frac{\partial E}{\partial w} = \frac{2}{n}\sum_{i=1}^{n}\left(y_{pr(i)} - y_{th(i)}\right).x_i$$

$$\frac{\partial E}{\partial b} = \frac{2}{n}\sum_{i=1}^{n}\left(y_{pr(i)} - y_{th(i)}\right)$$

```
for e in range(N): #en imposant un nombre de passages
    dw=0
    db=0
    for i in range(n): #somme
        y_i=w*x[i]+b #y prédit
        dw= dw+(y_i-y[i])*x[i]
        db=db+(y_i-y[i])
    dw=2*dw/n
    db=2*db/n
```

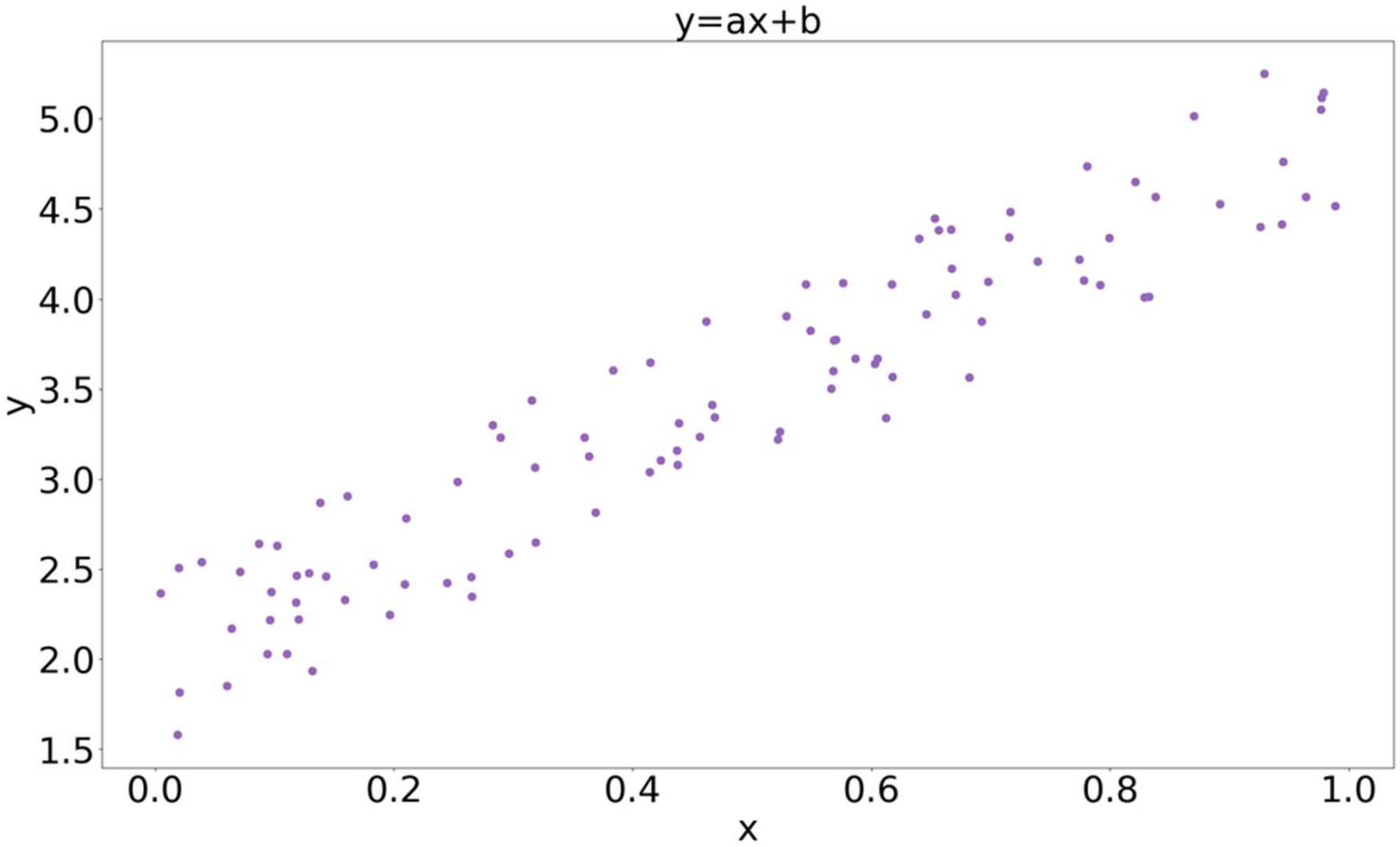
$$w_{N+1} = w_N - \mu\frac{\partial E}{\partial w}$$

$$b_{N+1} = b_N - \mu\frac{\partial E}{\partial b}$$

```
    #mise à jour des paramètres par descente du gradient
    w=w-μ*dw
    b=b-μ*db

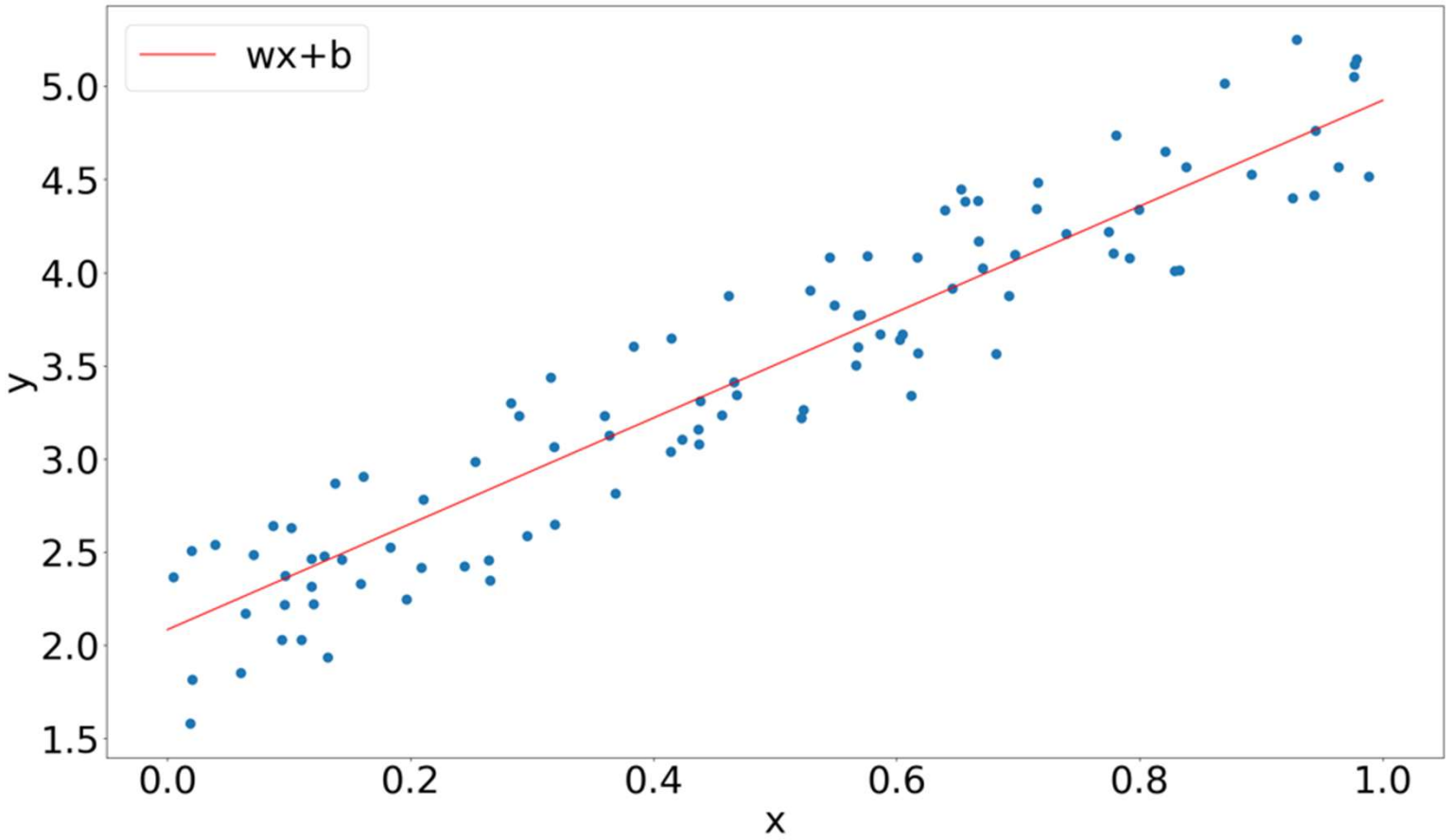
print('fin exécution (w,b)=(',w,',',b,')')
```

Générer un nuage de points



a=3
b=2
n=100

Régression linéaire



a=3
b=2
n=100

```
fin exécution (w,b)=( 2.8401912943305794 , 2.0838727682968576 )
```

Améliorations

```
#pour le coefficient de corrélation
def moyenne(Y):
    m=0
    for i in range(len(Y)):
        m+=Y[i]
    return m/len(Y)

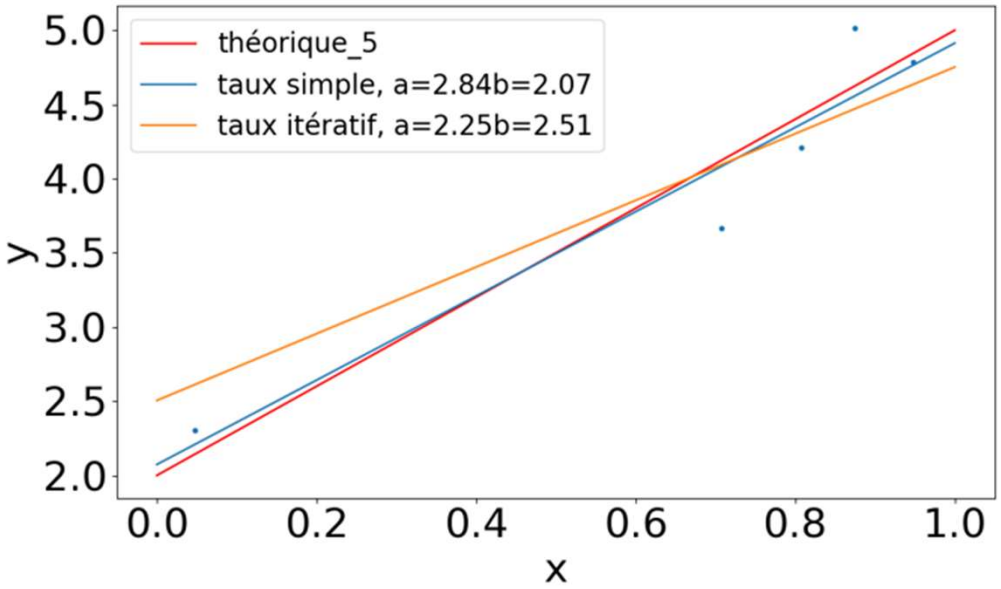
def coeff_Pearson(Y,X,w,b):
    S1=0
    S2=0
    for i in range(len(Y)):
        S1+= (Y[i]-(w*x[i]+b))**2
        S2+=(Y[i]-moyenne(Y))**2
    return 1-S1/S2
```

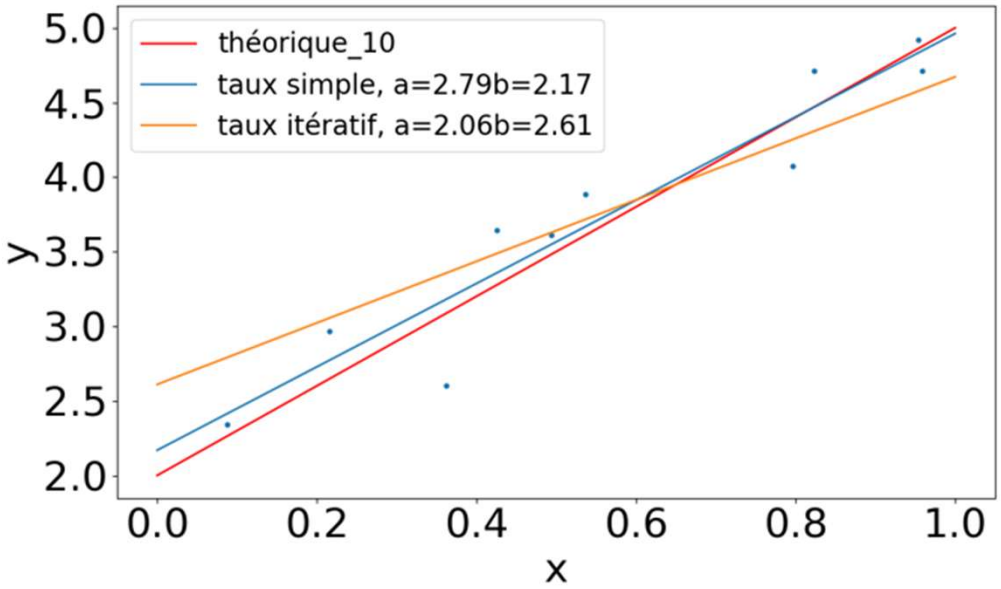
$$\bar{y} = \frac{1}{n}\sum_{i=1}^{n} y_{th(i)}$$

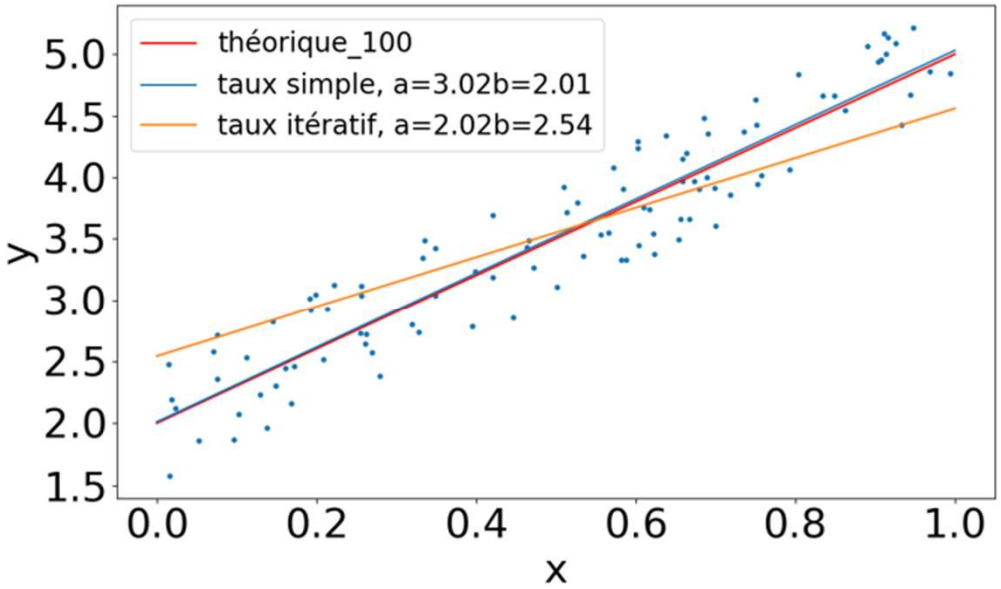
$$R^2 = 1 - \frac{\sum_{i=1}^{n}\left(y_{pr(i)} - y_{th(i)}\right)^2}{\sum_{i=1}^{n}\left(y_{th(i)} - \bar{y}\right)^2}$$

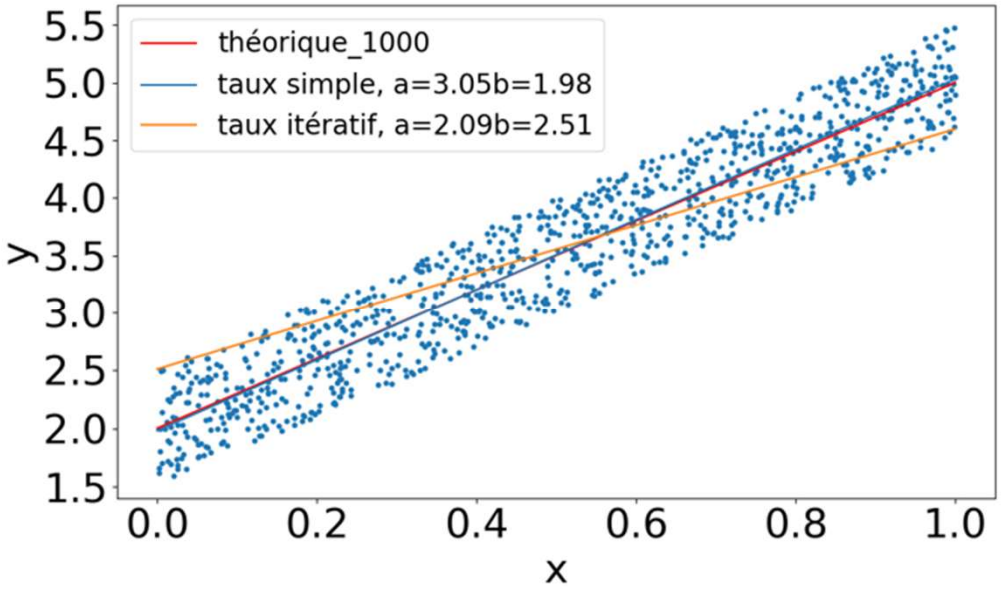
Exécution du programme :

```
taux simple 5 : 0.91
taux itératif 5 : 0.87
taux simple 10 : 0.90
taux itératif 10 : 0.84
taux simple 100 : 0.89
taux itératif 100 : 0.79
taux simple 1000 : 0.90
taux itératif 1000 : 0.81
```

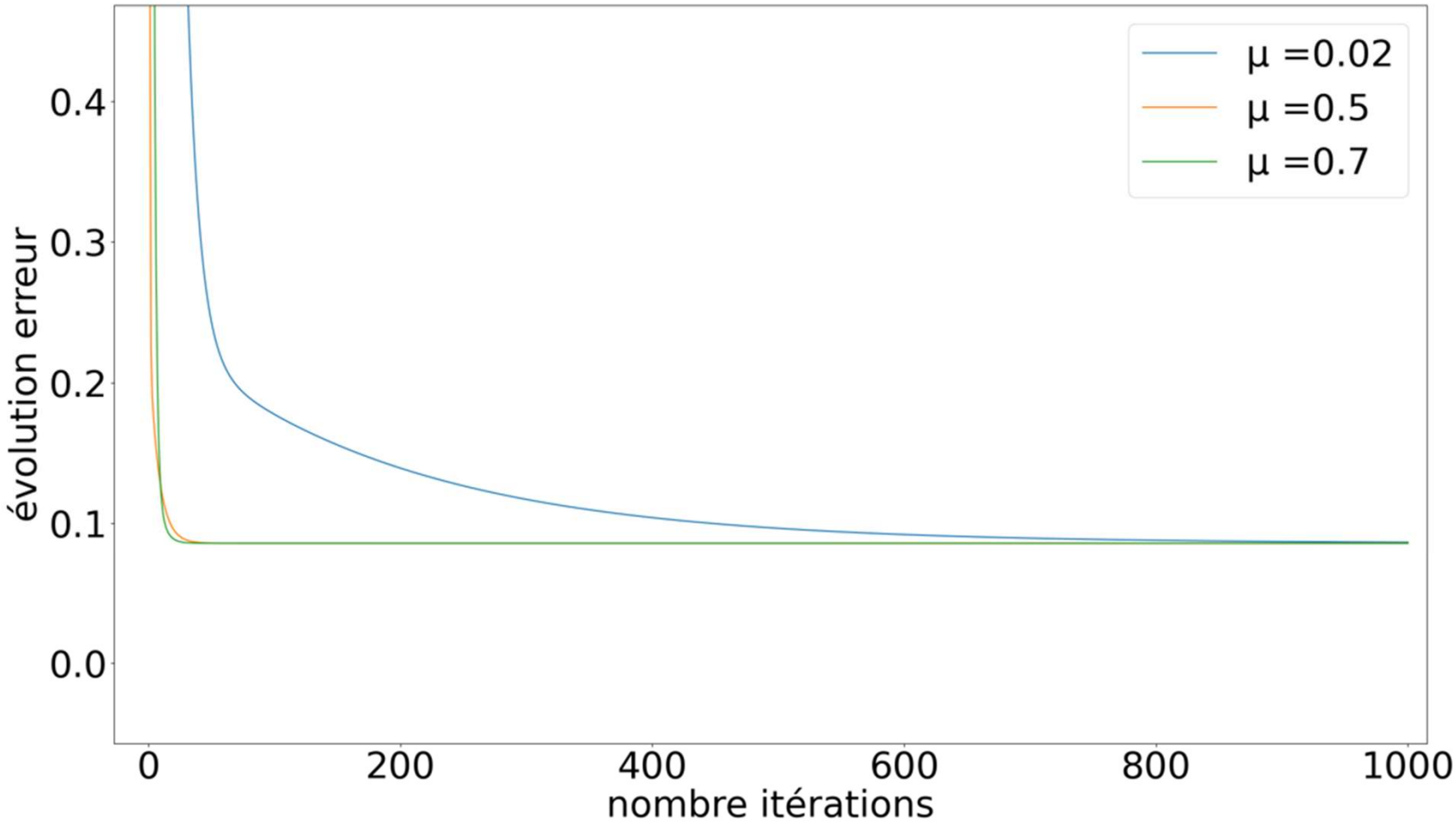

théorique_5
taux simple, a=2.84b=2.07
taux itératif, a=2.25b=2.51
y
x


théorique_10
taux simple, a=2.79b=2.17
taux itératif, a=2.06b=2.61
y
x

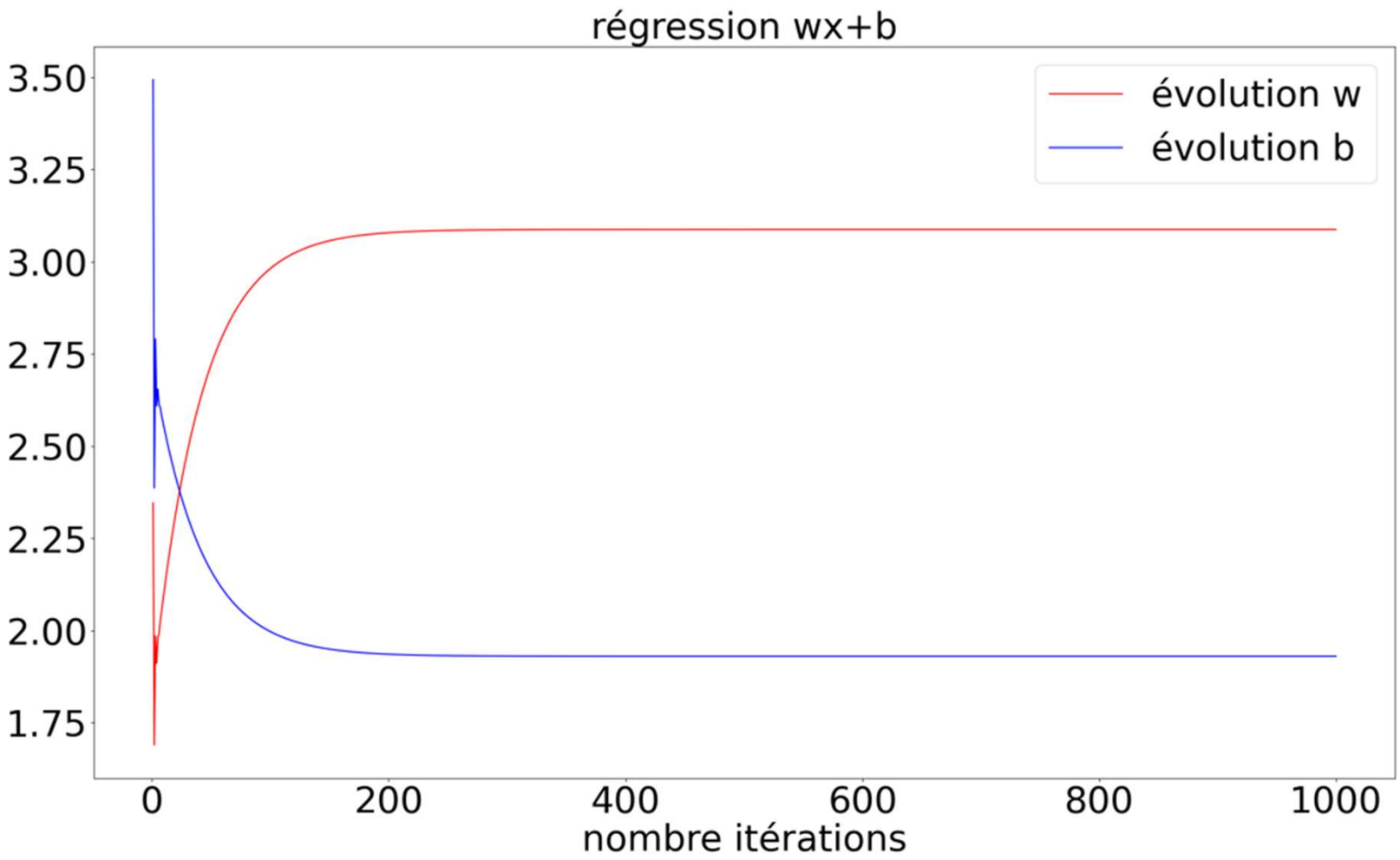

théorique_100
taux simple, a=3.02b=2.01
taux itératif, a=2.02b=2.54
y
x


théorique_1000
taux simple, a=3.05b=1.98
taux itératif, a=2.09b=2.51
y
x

Taux d’apprentissage



Nombre d’itérations



Application concrète

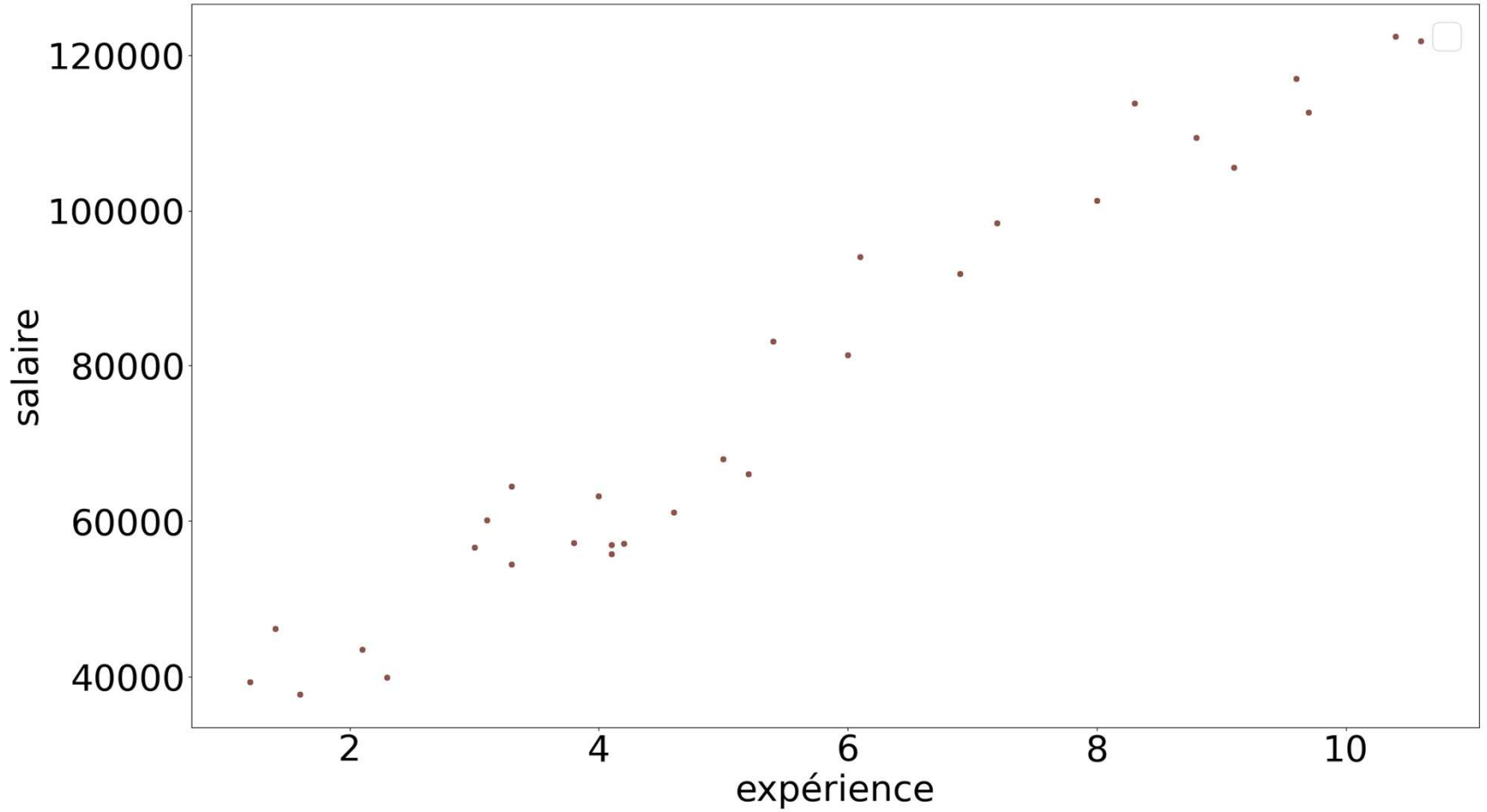
```
indice,YearsExperience,Salary
0,1.2000000000000002,39344.0
1,1.4000000000000001,46206.0
2,1.6,37732.0
3,2.1,43526.0
4,2.3000000000000003,39892.0S
5,3.0,56643.0
6,3.1,60151.0
7,3.3000000000000003,54446.0
8,3.3000000000000003,64446.0
9,3.8000000000000003,57190.0
10,4.0,63219.0
11,4.1,55795.0
12,4.1,56958.0
```

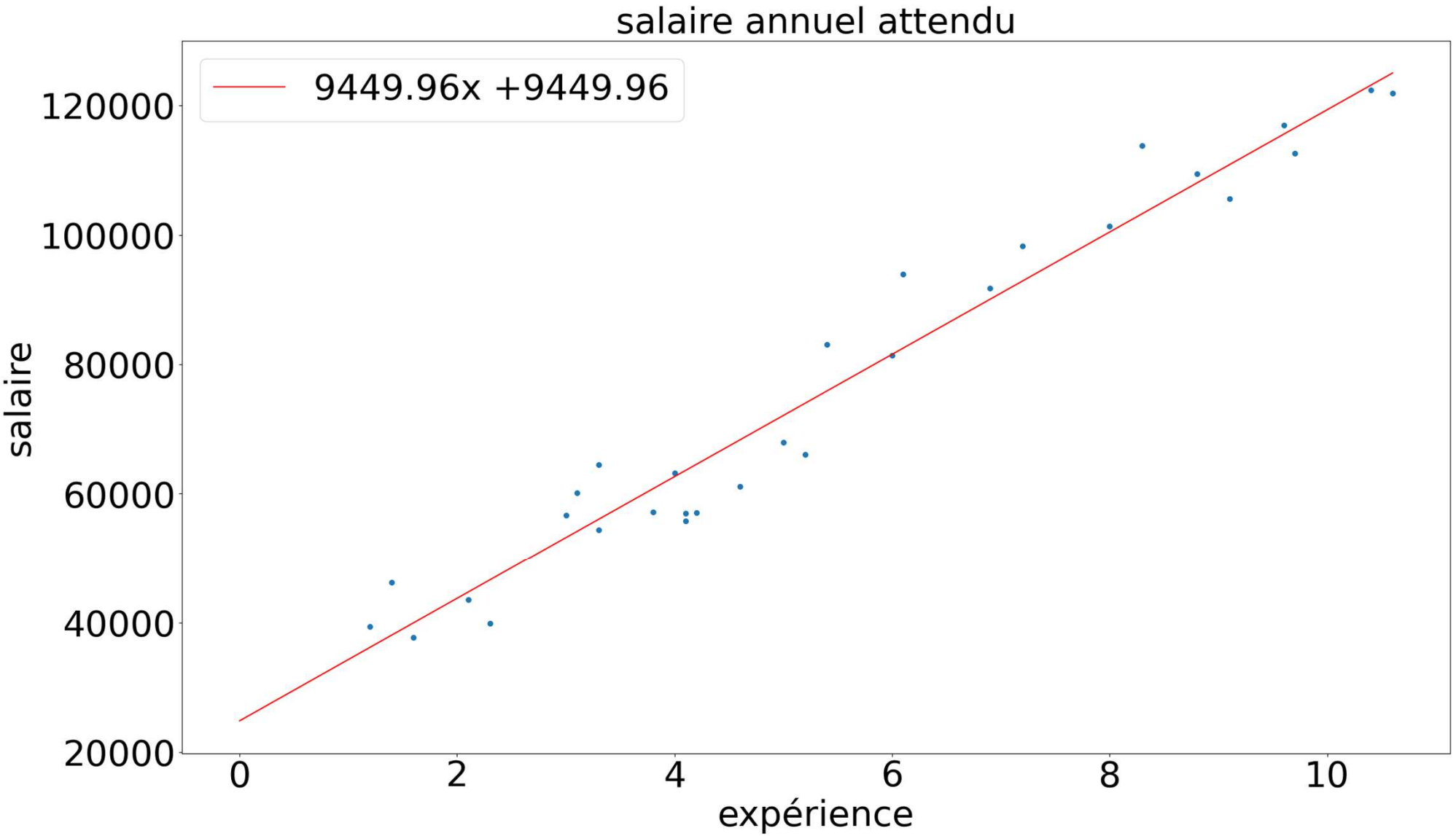
	indice	YearsExperience	Salary
0	0	1.2	39344.0
1	1	1.4	46206.0
2	2	1.6	37732.0
3	3	2.1	43526.0
4	4	2.3	39892.0
5	5	3.0	56643.0
6	6	3.1	60151.0
7	7	3.3	54446.0
8	8	3.3	64446.0
9	9	3.8	57190.0
10	10	4.0	63219.0
11	11	4.1	55795.0
12	12	4.1	56958.0

```
dossier= pd.read_csv(chemin, decimal="." )
```

```
x = dossier['YearsExperience'].tolist()
y = dossier['Salary'].tolist()


#générer le nuage de point
plt.scatter(x,y,s=30)
plt.xlabel('expérience',fontsize=40)
plt.ylabel('salaire',fontsize=40)
plt.xticks(fontsize=40)
plt.yticks(fontsize=40)
plt.show()
```




salaire annuel attendu
9449.96x +9449.96
120000
100000
80000
60000
40000
20000
salaire
0
2
4
6
8
10
expérience

Limites du modèle

Paramètres d'apprentissage :

- Données d'apprentissage
- Taux d'apprentissage
- Nombre d'itérations

programmation :

- Utilisation de listes
- Programmation en dimension n ?

Passage à une dimension supérieure
$ax + by + c$

Données d'apprentissage :

$$(x_i, y_i, z_i)_{i \in [\![1,n]\!]}$$

Coefficient synaptique ou poids :

$$[a, b, c]$$

Fonction d'activation :

f = id

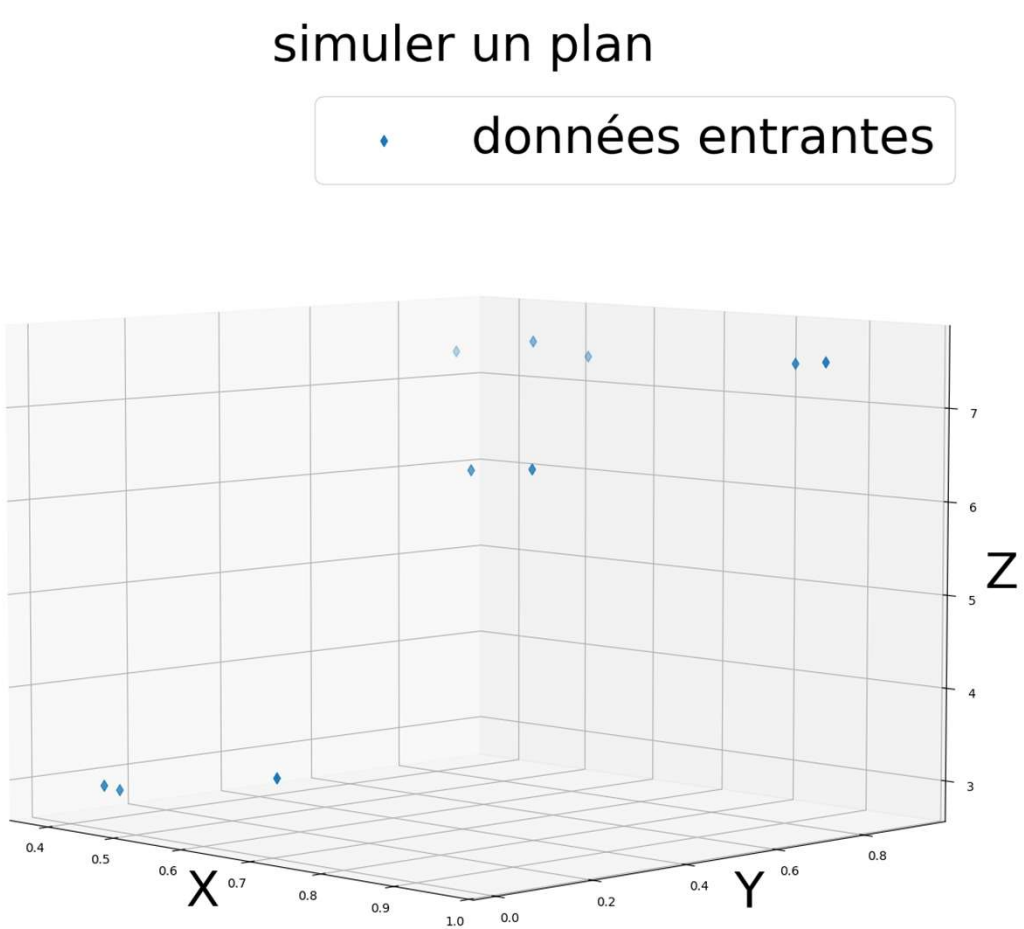
Modèle prédit :

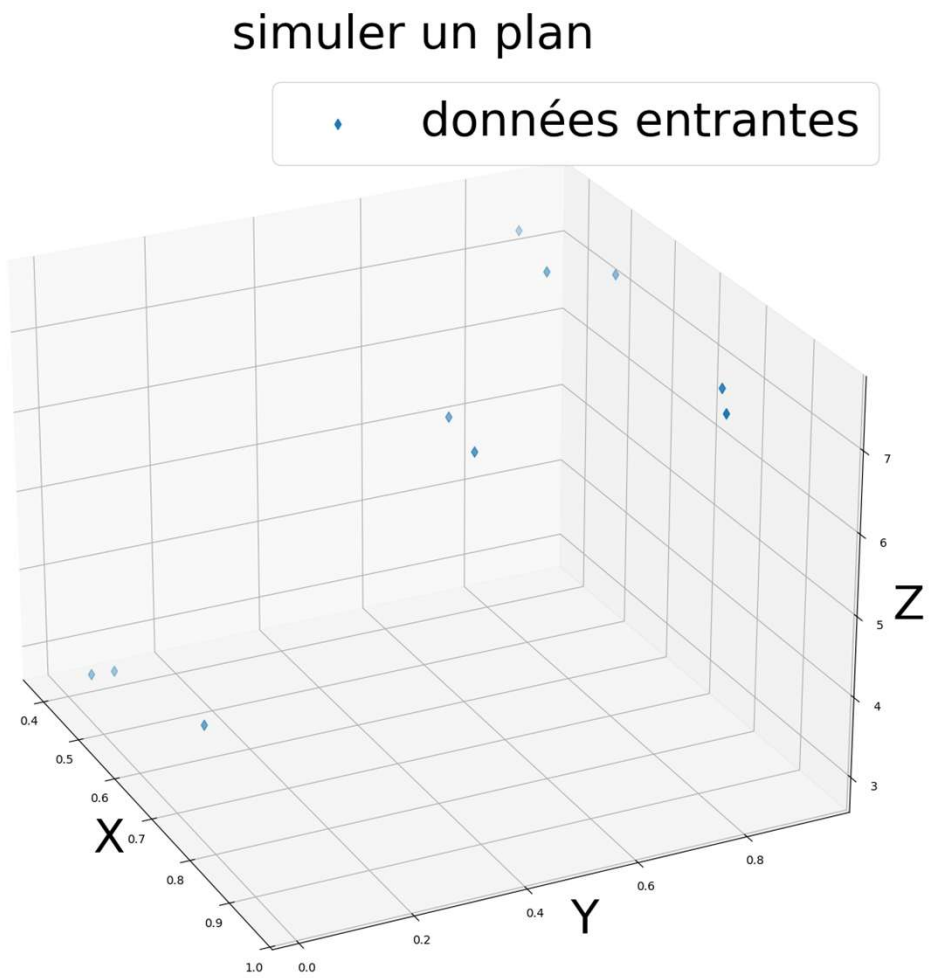
$$z_{pr} = \begin{pmatrix} ax_1 + by_1 + c \\ \vdots \\ ax_n + by_n + c \end{pmatrix}$$

Modèle théorique :

$$z_{th} = \begin{pmatrix} z_1 \\ \vdots \\ z_n \end{pmatrix}$$

Générer un nuage des points





$x = [liste\ d'abscisses]$ $y = [liste\ d'ordonnées]$

$$w = \begin{pmatrix} a \\ b \\ c \end{pmatrix}$$

```
#générer des poids aléatoires
np.random.seed(0)
a= np.random.random()
b= np.random.random()
c = np.random.random()
w=np.array([a,b,c])
print(w)
```

$$v = \begin{pmatrix} x_1 & y_1 & 1 \\ \vdots & \vdots & \vdots \\ x_n & y_n & 1 \end{pmatrix}$$

```
e=[1 for i in range(n)]
v=np.array([[x[i],y[i],e[i]]for i in range(n)])
w_th=np.array([a_th,b_th,c_th])

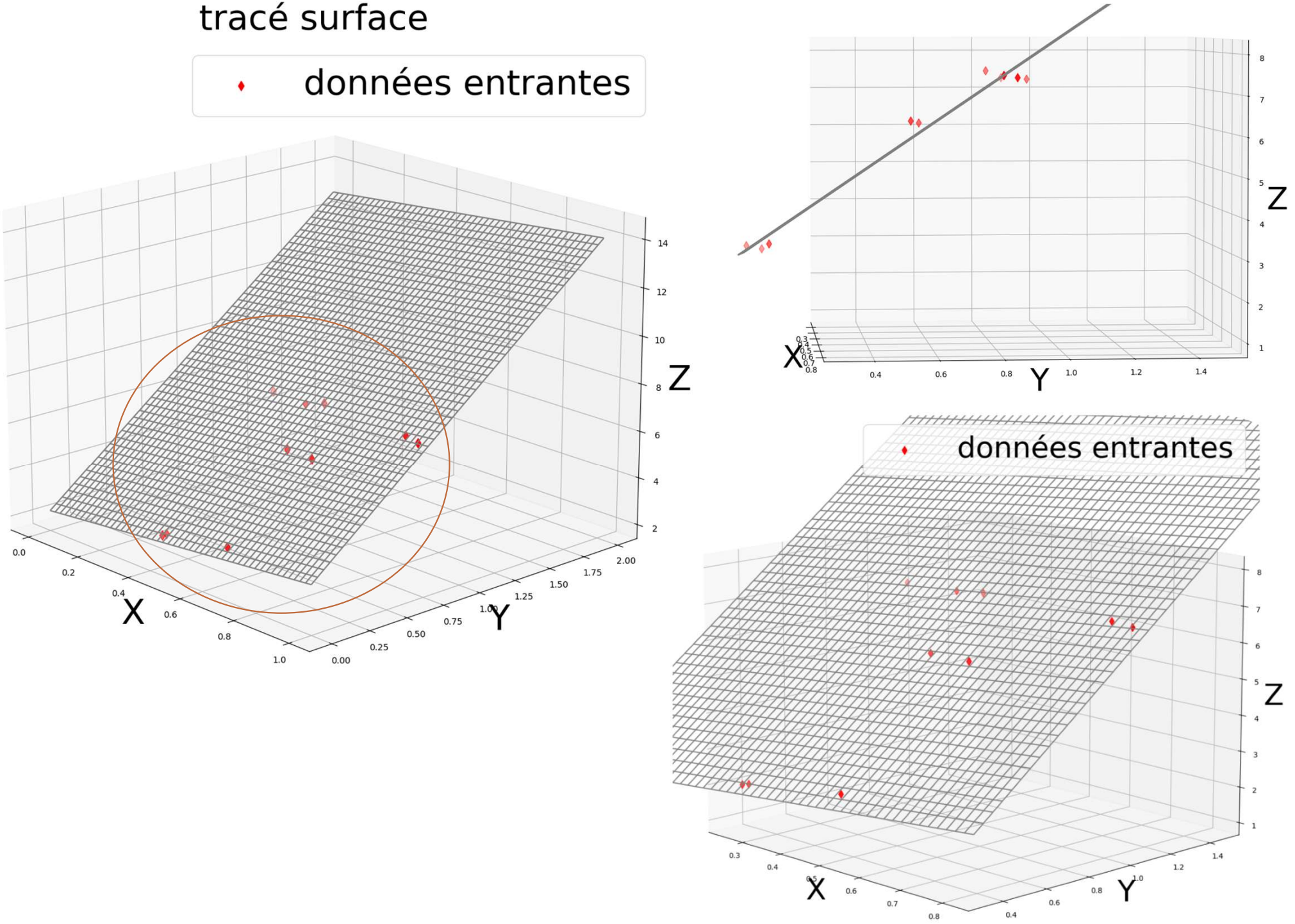
##descente du gradient

N=100 #nbr de fois que le programme est entrainé
μ=0.05  #taux d'apprentissage
for e in range(N):
    z_pr=np.array(v@w) #@ produit matriciel
    dw=-2*np.array((v.T)@(z-z_pr)) #.T transposée
    w=np.array(w-μ*dw)
```

$$z_{pr} = \begin{pmatrix} ax_1 + by_1 + c \\ \vdots \\ ax_n + by_n + c \end{pmatrix}$$

$$z = \begin{pmatrix} z_1 \\ \vdots \\ z_n \end{pmatrix}$$

tracé surface



ANNEXES

1) Régression linéaire (1 dimension)
   i. Générer le nuage de point (24)
   ii. Algorithme d'apprentissage (25)
   iii. Résultats de l'apprentissage (26)
   iv. Coefficient de corrélation (27)
   v. Comparaison des taux d'apprentissage (28-29)
   vi. Évolution de l'erreur (30)
   vii. Évolution des paramètres (31)
   viii. Application concrète (32-33)

2) Régression dans le plan (2 dimensions)
   i. Générer le nuage de points (34)
   ii. Algorithme d'apprentissage (35)
   iii. Résultats de l'apprentissage (36)

```python
import numpy as np
import matplotlib.pyplot as plt
import random

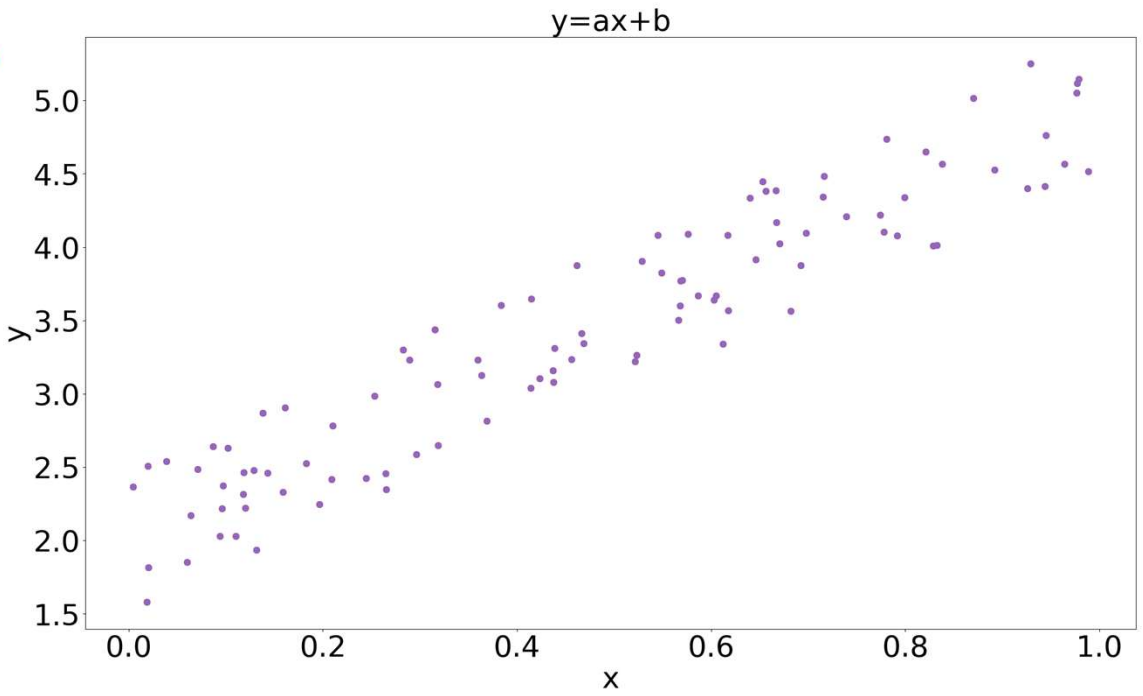
##différentes commandes spéficiques
#.tolist transforme un tableau (ici 1 ligne, n colonnes) en liste
#le np.random.seed(graine) permet de faire la même simulation jusqu'à changer la graine

a=3 # coeff directeur
b=2 #ordonnée à l'origine


n=100 #nbr de point ie données entrantes
np.random.seed(0) #pour avoir la même simulation à chaque exécution du programme
x = np.random.rand(n).tolist() #génère une liste de n flottants compris entre 0 et 1

y =[ b + a * x[i] + np.random.rand(len(x)).tolist()[i]-0.5 for i in range(len(x))]
#on retranche 0.5 car en moyenne le np.random.rand() nous décale en ordonnée y +0.5


#tracer le graphique (générer le nuage de points)
plt.scatter(x,y,s=70)
plt.xlabel('x',fontsize=40)
plt.ylabel('y',fontsize=40)
plt.title(label='y=ax+b',fontsize=40)
#taille de la police
plt.xticks(fontsize=40)
plt.yticks(fontsize=40)
plt.show()
```



```
#modèle non entrainé
np.random.seed(0)
w=np.random.random()
b=np.random.random()
print('paramètres générés (w,b)=(',w,',',b,')')



N=1000 #nbr de fois que le programme est entrainé
μ=0.02  #taux d'apprentissage

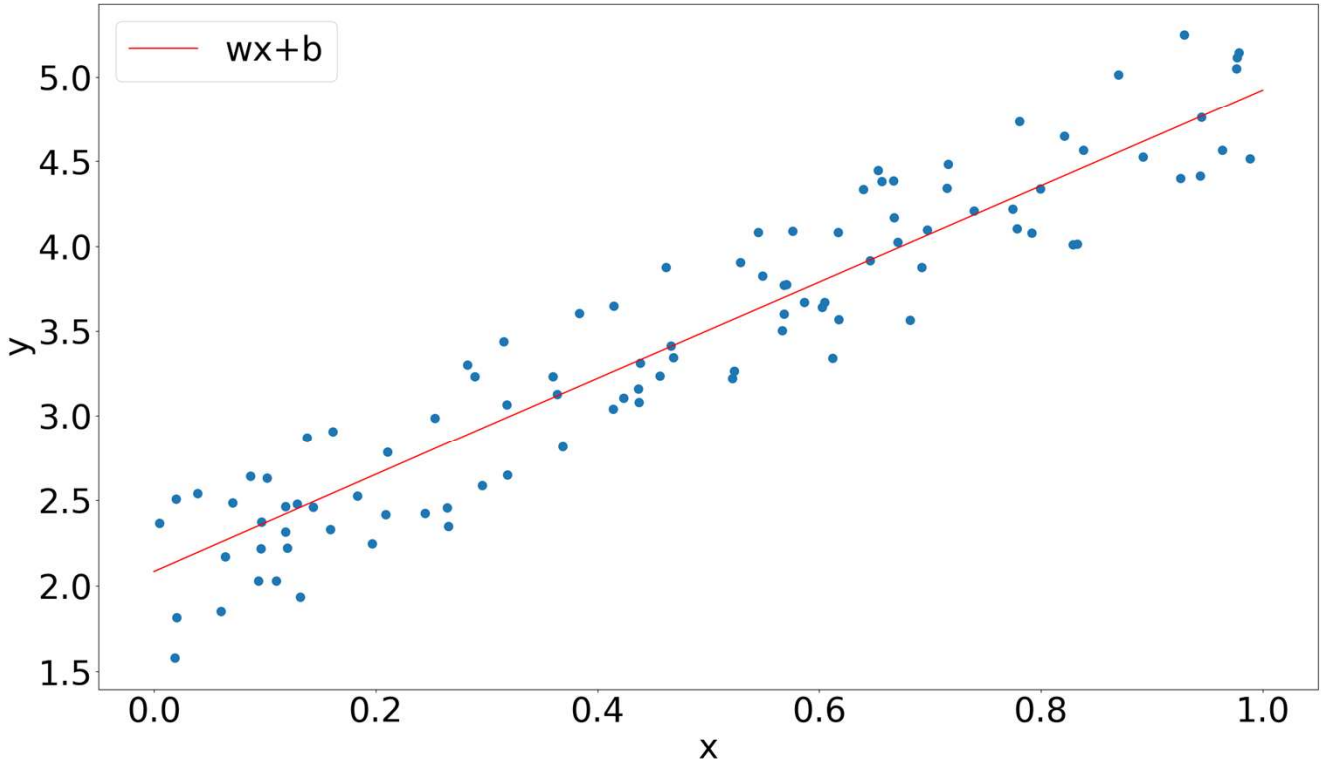
for e in range(N): #en imposant un nombre de passages
    dw=0
    db=0
    for i in range(n): #somme
        y_i=w*x[i]+b #y prédit
        dw= dw+(y_i-y[i])*x[i]
        db=db+(y_i-y[i])
    dw=2*dw/n
    db=2*db/n

    #mise à jour des paramètres par descente du gradient
    w=w-μ*dw
    b=b-μ*db

print('fin exécution (w,b)=(',w,',',b,')')
```

```
paramètres générés (w,b)=( 0.5488135039273248 , 0.7151893663724195 )
fin exécution (w,b)=( 2.8401912943305794 , 2.0838727682968576 )
```

```
plt.close('all')
X=np.linspace(0,1) #points d'abscisse répartis sur [0,1]
Y=w*X+b
plt.scatter(x,y,s=100)
plt.plot(X,Y, color='r',label='wx+b')
plt.xlabel('x',fontsize=40)
plt.ylabel('y',fontsize=40)
plt.xticks(fontsize=40)
plt.yticks(fontsize=40)
plt.legend(fontsize=40)
plt.show()
```



```python
def erreur(w,b): #y liste de donnés (points), yth modèle théorique, n nbr de points
    E=0

    for i in range(1,n):
        E+= (w*x[i]+b - y[i])**2
    return E/n

def grad(dw,db): #norme du gradient
    return np.sqrt(dw**2+ db**2)

#pour le coefficient de corrélation
def moyenne(Y):
    m=0
    for i in range(len(Y)):
        m+=Y[i]
    return m/len(Y)

def coeff_Pearson(Y,X,w,b):
    S1=0
    S2=0
    for i in range(len(Y)):
        S1+= (Y[i]-(w*x[i]+b))**2
        S2+=(Y[i]-moyenne(Y))**2
    return 1-S1/S2
```

```
taux simple 5 : 0.91
taux itératif 5 : 0.87
taux simple 10 : 0.90
taux itératif 10 : 0.84
taux simple 100 : 0.89
taux itératif 100 : 0.79
taux simple 1000 : 0.90
taux itératif 1000 : 0.81
```

```
def perceptron_simple(x,y,w,b,N):
# liste de données d'apprentissage + liste de sorties théorique, N nbre de passage
    L=[]
    μ_0=0.5  #taux d'apprentissage
    for e in range(1,N+1):
        dw=0
        db=0
        μ= μ_0
        for i in range(n):
            y_i=w*x[i]+b #y prédit
            dw= dw+(y_i-y[i])*x[i]
            db=db+(y_i-y[i])
        dw=2*dw/n
        db=2*db/n

#mise à jour des paramètres par descente de gradient
        w=w-μ*dw
        b=b-μ*db
    L.append(w)
    L.append(b)
    return L
```

Taux simple

Taux itératif

```
def perceptron_lin(x,y,w,b,N):
    L=[]
    μ_0=0.5  #taux d'apprentissage
    for e in range(1,N+1):
        dw=0
        db=0
        μ= μ_0/(e+1)
        for i in range(n):
            y_i=w*x[i]+b #y prédit
            dw= dw+(y_i-y[i])*x[i]
            db=db+(y_i-y[i])
        dw=2*dw/n
        db=2*db/n

#mise à jour des paramètres par descente de gradient
        w=w-μ*dw
        b=b-μ*db
    L.append(w)
    L.append(b)
    return L
```

```
'''
Comparaison des taux d'apprentissage
'''
a=3
o=2

X=np.linspace(0,1)
Y_th= a*X+o

N=[5, 10, 100,1000]

p1, p2=100,1000 #nombre d'itérations

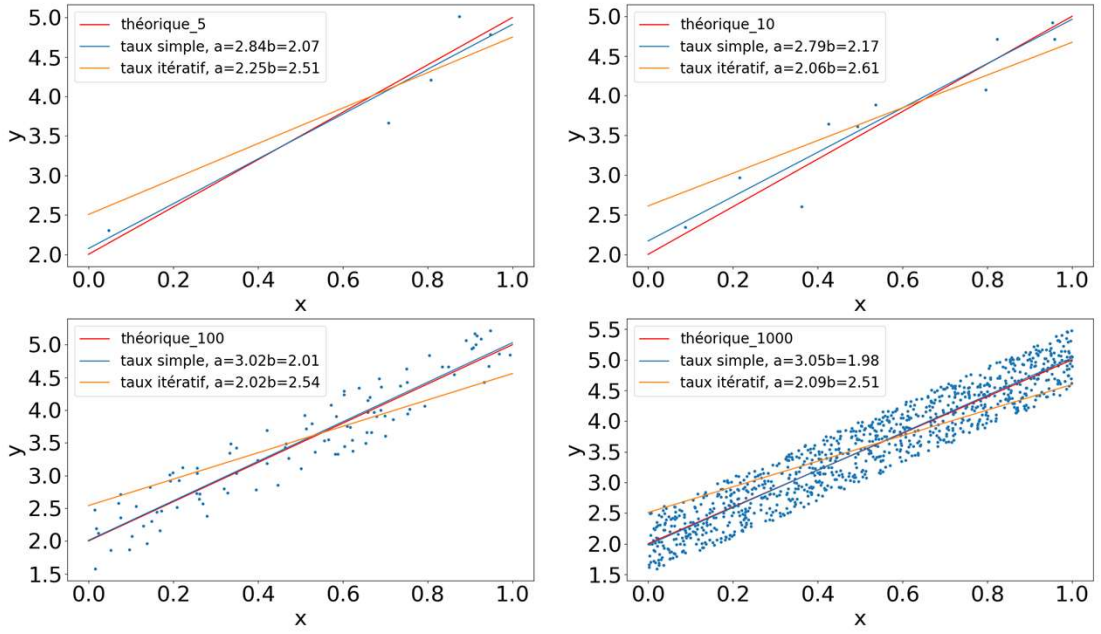
plt.close('all')
for i in range(len(N)):
    n=N[i] #nbr de points

    plt.subplot((len(N)+1)//2, 2, i+1)
    x = np.random.rand(n).tolist() #génère une liste de n flottants compris entre 0 et 1
    y =[ o + a * x[i] + np.random.rand(len(x)).tolist()[i]-0.5 for i in range(len(x))]
    plt.scatter(x,y,s=10) #(prend en argument deux listes)

    plt.plot(X,Y_th, label='théorique_' +str(n), color='r')
    Y_simple= perceptron_simple(x,y,w,b,p1)[0]*X+perceptron_simple(x,y,w,b,p1)[1]
    plt.plot(X,Y_simple, label='taux simple, a='+str("{:.2f}".format(perceptron_simple(x,y,w,b,p1)[0]))
             + 'b='+ str("{:.2f}".format(perceptron_simple(x,y,w,b,p1)[1])))
    Y_lin= perceptron_lin(x,y,w,b,p2)[0]*X+perceptron_lin(x,y,w,b,p2)[1]
    plt.plot(X,Y_lin, label='taux itératif, a='+str("{:.2f}".format(perceptron_lin(x,y,w,b,p2)[0]))
             + 'b='+ str("{:.2f}".format(perceptron_lin(x,y,w,b,p2)[1])))

    print('taux simple',N[i], ':', str("{:.2f}".format( coeff_Pearson(y,x,perceptron_simple(x,y,w,b,p1)[0]
                                                                    ,perceptron_simple(x,y,w,b,p1)[1]))))
    print('taux itératif',N[i],':' ,str("{:.2f}".format(coeff_Pearson(y,x,perceptron_lin(x,y,w,b,p2)[0]
                                                                    ,perceptron_lin(x,y,w,b,p2)[1]))))

    plt.xticks(fontsize=30)
    plt.yticks(fontsize=30)
    plt.xlabel('x',fontsize=30)
    plt.ylabel('y',fontsize=30)
        plt.legend(fontsize=20)
    plt.show()
```



```python
#modèle non entrainé
np.random.seed(0)
w=np.random.random()
b=np.random.random()
print(w,b)

def erreur(w,b):
    E=0
    for i in range(1,n):
        E+= (w*x[i]+b - y[i])**2
    return E/n

N=1000 #nombre d'itérations du programme

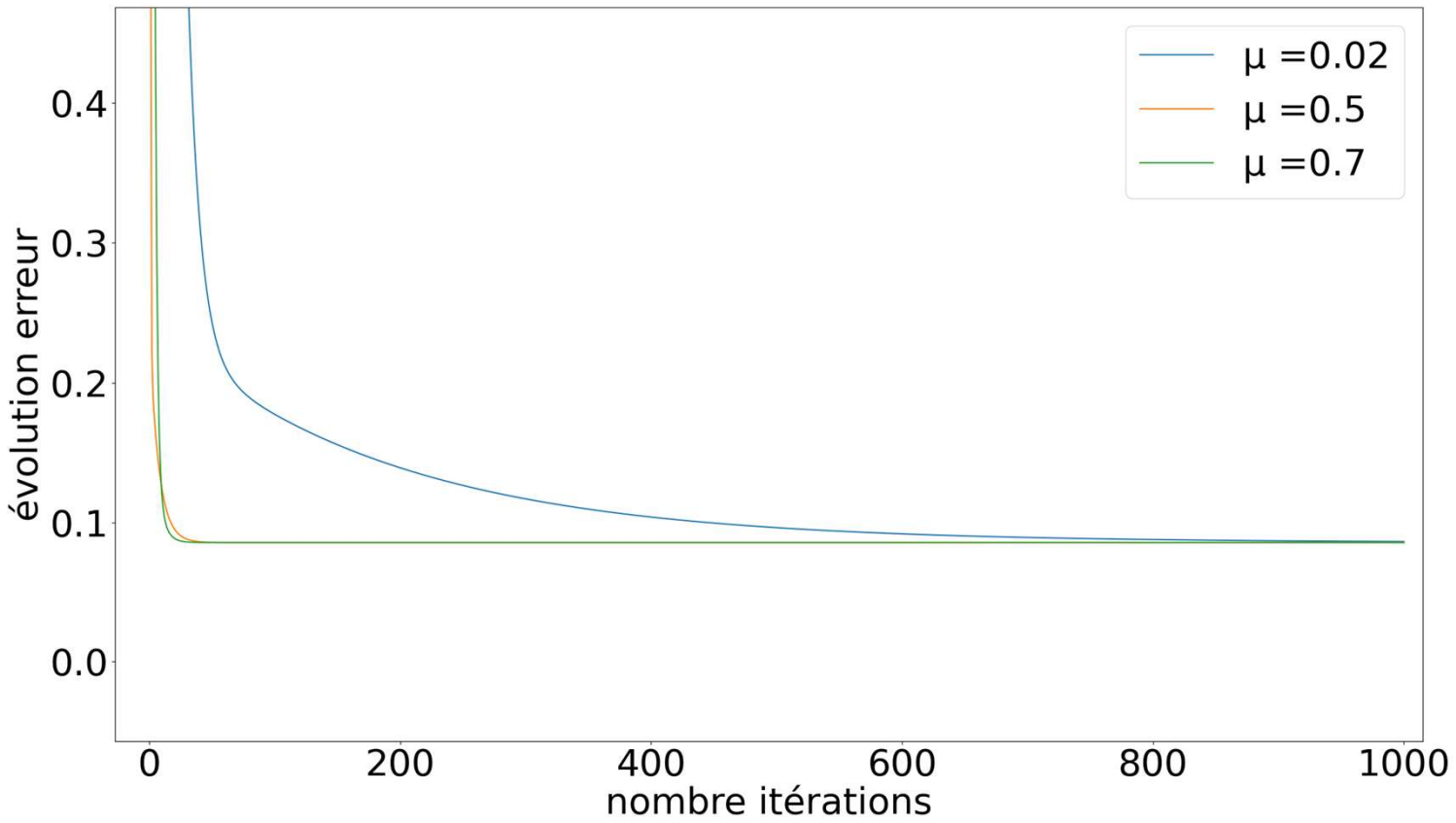
def perceptron_w(x,y,μ,w,b):
    L_w=[]
    for e in range(1,N+1):
        dw=0
        for i in range(n): #somme
            y_i=w*x[i]+b #y prédit
            dw= dw+(y_i-y[i])*x[i]
        dw=2*dw/n
        w=w-μ*dw
        L_w.append(w)
    return L_w

def perceptron_b(x,y,μ,w,b):
    L_b=[]
    for e in range(1,N+1):
        db=0
        for i in range(n): #somme
            y_i=w*x[i]+b #y prédit
            db=db+(y_i-y[i])
        db=2*db/n
        b=b-μ*db
        L_b.append(b)

    return L_b

#tracé de l'évolution de l'erreur suivant différents taux d'apprentissage
plt.close('all')

μ=[0.02, 0.5, 0.7]
X=[i for i in range(1,N+1)]
for k in range(len(μ)):
    Y=[erreur(perceptron_w(x,y,μ[k],w,b)[i],perceptron_b(x,y,μ[k],w,b)[i])for i in range(N)]
    plt.plot(X,Y, label='μ ='+str(μ[k]))
    plt.xlabel('nombre itérations',fontsize=40)
    plt.ylabel('évolution erreur',fontsize=40)
#plt.title('évolution erreur',fontsize=40)
plt.xticks(fontsize=40)
plt.yticks(fontsize=40)
plt.legend(fontsize=40)
plt.show()
```



```
N=1000 #nombre d'itérations du programme
L_w=[]
L_b=[]
for e in range(1,N+1):

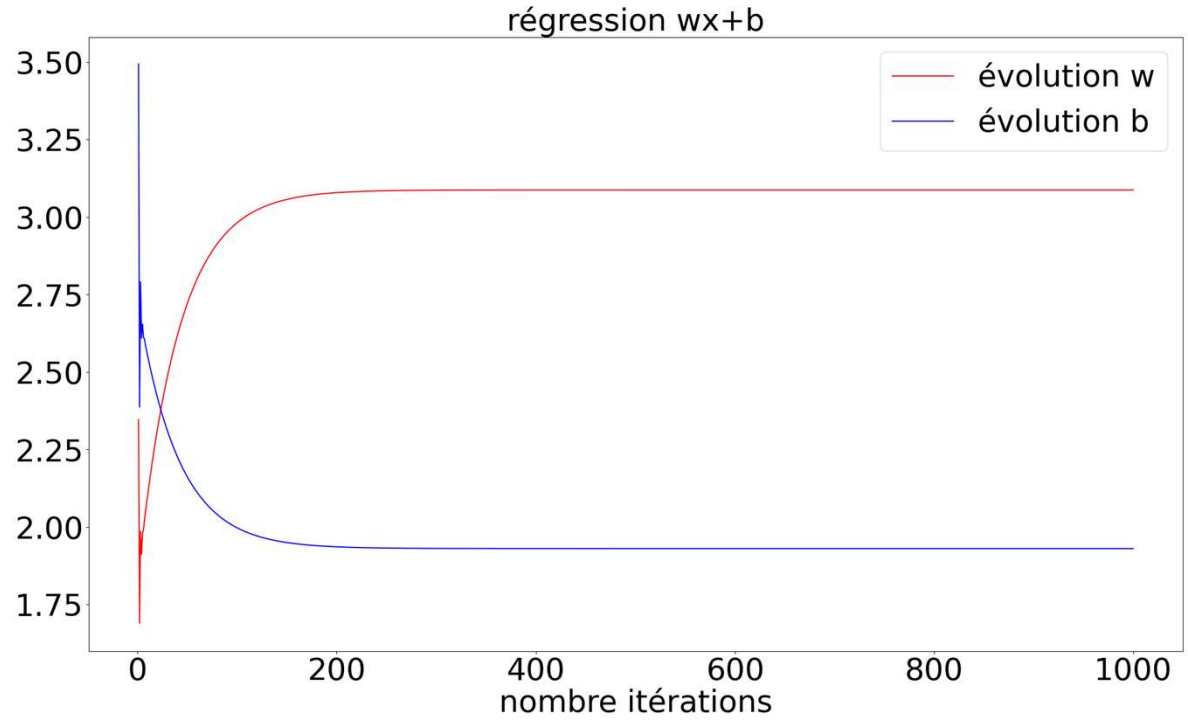
    dw=0
    db=0
    μ= μ_0
    for i in range(n): #somme
        y_i=w*x[i]+b #y prédit
        dw= dw+(y_i-y[i])*x[i]
        db=db+(y_i-y[i])
    dw=2*dw/n #on obtient la dérivée de l'erreur par rapport à w
    db=2*db/n

    #mise à jour des paramètres par descente de gradient
    w=w-μ*dw
    b=b-μ*db
    L_w.append(w)
    L_b.append(b)

##tracé de l'évolution du poids et de b

plt.close('all')
#tracé de l'évolution du poids
X=[i for i in range(1,N+1)]
Y=L_w
Z=L_b
plt.title('régression wx+b',fontsize=40)
plt.plot(X,Y, color='r', label='évolution w')
plt.plot(X,Z, color='b', label='évolution b')
plt.xticks(fontsize=40)
plt.yticks(fontsize=40)
plt.xlabel('nombre itérations',fontsize=40)
#plt.ylabel('',fontsize=40)
plt.legend(fontsize=40)
plt.show()
```

Droite de régression : 3x+2



```
'''
ESSAI APPLICATION linéraire
'''


import matplotlib.pyplot as plt
import pandas as pd

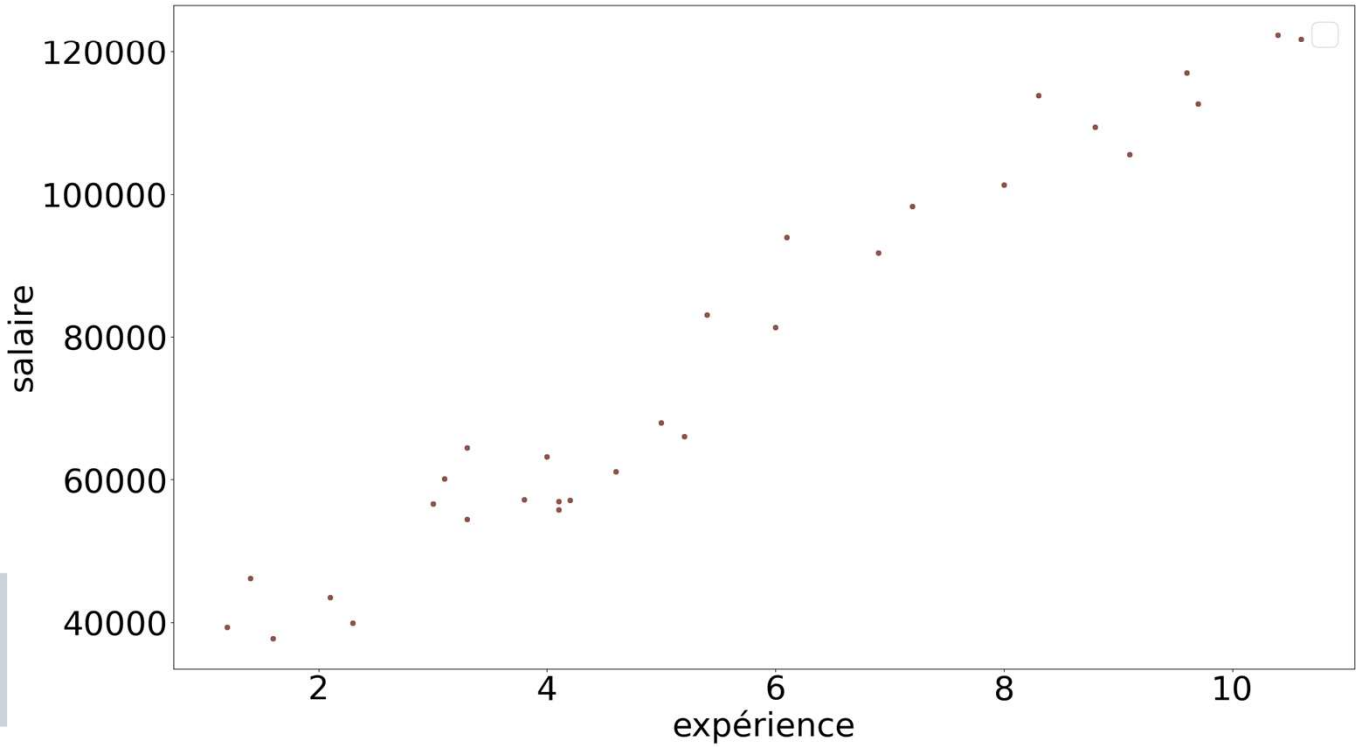

chemin = 'C:/Users/Julaivane/OneDrive/PREPA/SPE/TIPE/dataset/Salary_dataset.csv'
dossier= pd.read_csv(chemin, decimal="." )
print(dossier)


L_salaire=dossier['Salary'].tolist()
L_exp=dossier['YearsExperience'].tolist()


x = dossier['YearsExperience'].tolist()
y = dossier['Salary'].tolist()


#générer le nuage de point
plt.scatter(x,y,s=30)
plt.xlabel('expérience',fontsize=40)
plt.ylabel('salaire',fontsize=40)
plt.xticks(fontsize=40)
plt.yticks(fontsize=40)
plt.show()
```

	indice	YearsExperience	Salary
0	0	1.2	39344.0
1	1	1.4	46206.0
2	2	1.6	37732.0
3	3	2.1	43526.0
4	4	2.3	39892.0
5	5	3.0	56643.0
6	6	3.1	60151.0
7	7	3.3	54446.0
8	8	3.3	64446.0
9	9	3.8	57190.0
10	10	4.0	63219.0
11	11	4.1	55795.0
12	12	4.1	56958.0



Dataset :

https://www.kaggle.com/datasets/abhishek14398/salary-dataset-simple-linear-regression

```
#modèle non entrainé

w=np.random.random()
b=np.random.random()
print(w,b)

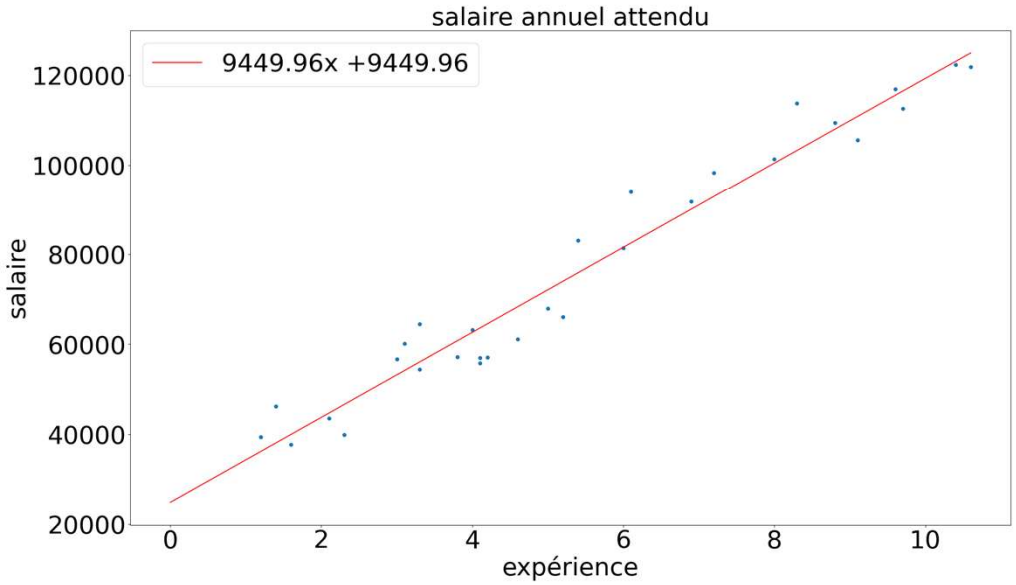


n=len(x)
N=10000 #nbr de fois que le programme est entrainé
μ=0.02  #taux d'apprentissage
for e in range(N):
    dw=0
    db=0
    for i in range(n): #somme
        y_i=w*x[i]+b #y prédit
        dw= dw+(y_i-y[i])*x[i]
        db=db+(y_i-y[i])
    dw=2*dw/n
    db=2*db/n

    #mise à jour des paramètres par descente de gradient

    w=w-μ*dw
    b=b-μ*db


##tracé du graphique
plt.close('all')
X=np.linspace(0,x[len(x)-1])
Y=w*X+b
plt.scatter(x,y,s=30)
plt.plot(X,Y, color='r', label=str("{:.2f}".format(w))+'x +'+str("{:.2f}".format(w)))
plt.title('salaire annuel attendu',fontsize=40)
plt.xticks(fontsize=40)
plt.yticks(fontsize=40)
plt.xlabel('expérience',fontsize=40)
plt.ylabel('salaire',fontsize=40)
plt.legend(fontsize=40)
plt.show()
```

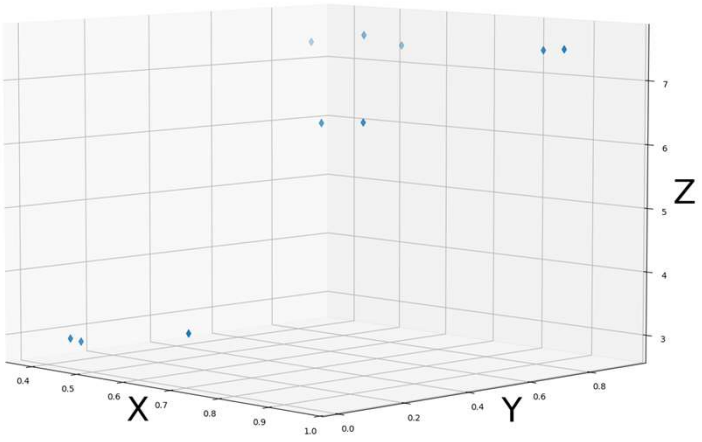


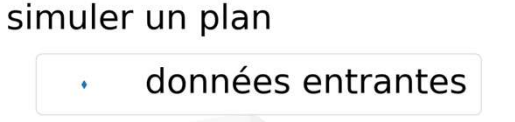



```
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import axes3d  # Fonction pour la 3D
from matplotlib import cm
from matplotlib.ticker import LinearLocator, FormatStrFormatter
import numpy as np

##générer un plan de points
a_th,b_th,c_th = 2,5,2
n=10 #nbr de point ie données entrantes
np.random.seed(0)
x =np.random.rand(n).tolist()
y =np.random.rand(n).tolist()

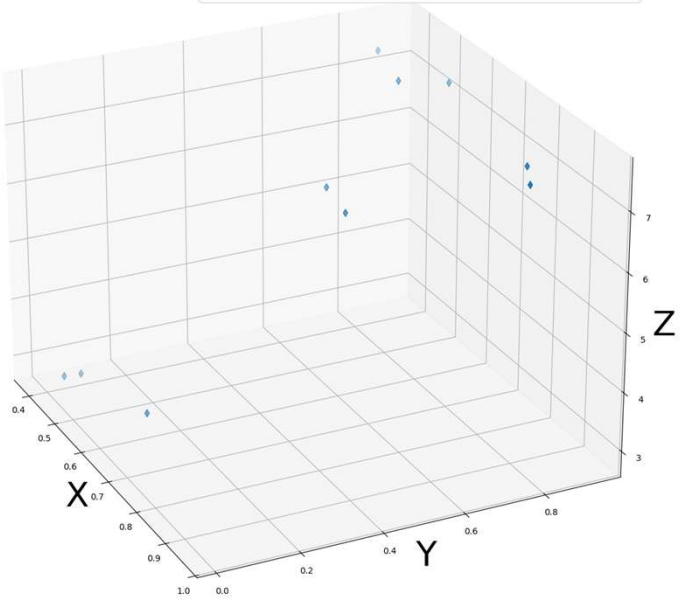
z =[a_th * x[i] + np.random.rand(len(x)).tolist()[i]-0.5 +b_th*y[i]+c_th for i in range(len(x))]
```






```
plt.close('all')

#Tracé du résultat en 3D
fig = plt.figure()
ax = fig.add_subplot(projection = '3d')
ax.scatter(x, y, z, label='données entrantes', marker='d',s=40)
plt.title("simuler un plan",fontsize=40)
ax.set_xlabel('X',fontsize=40)
ax.set_ylabel('Y',fontsize=40)
ax.set_zlabel('Z',fontsize=40)
plt.legend(fontsize=40)
plt.tight_layout()
plt.show()
```



```python
#générer des poids aléatoires
np.random.seed(0)
a= np.random.random()
b= np.random.random()
c = np.random.random()
w=np.array([a,b,c])
print('modèle non entrainé:',w)


e=[1 for i in range(n)]
v=np.array([[x[i],y[i],e[i]]for i in range(n)])
w_th=np.array([a_th,b_th,c_th])

##descente du gradient

N=100 #nbr de fois que le programme est entrainé
μ=0.05  #taux d'apprentissage
for e in range(N):
    z_pr=np.array(v@w) #@ produit matriciel
    dw=-2*np.array((v.T)@(z-z_pr)) #.T transposée
    w=np.array(w-μ*dw)
print('paramètres théoriques:',w_th)
print('fin exécution:', w)
```

```
modèle non entrainé: [0.5488135  0.71518937 0.60276338]
paramètres théoriques: [2 5 2]
fin exécution: [1.26431052 5.28296514 2.22199131]
```

```
#tracer le plan de régression
import numpy as np
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D
ax=Axes3D(plt.figure())
W_list=w.tolist()

def f(x,y):
    return x*W_list[0]+y*W_list[1]+ W_list[2]


fig = plt.figure()
ax = fig.add_subplot(projection = '3d')
x_ax=np.linspace(0,1)
y_ay=np.linspace(0,2)
X,Y=np.meshgrid(x_ax,y_ay)
ax.scatter(x, y, z, label='données entrantes', marker='d', color='red', s=60)
plt.title("tracé surface",fontsize=40)
ax.plot_wireframe(X,Y,f(X,Y),color='grey')
ax.set_xlabel('X',fontsize=40)
ax.set_ylabel('Y',fontsize=40)
ax.set_zlabel('Z',fontsize=40)
plt.legend(fontsize=40)
plt.show()
```

